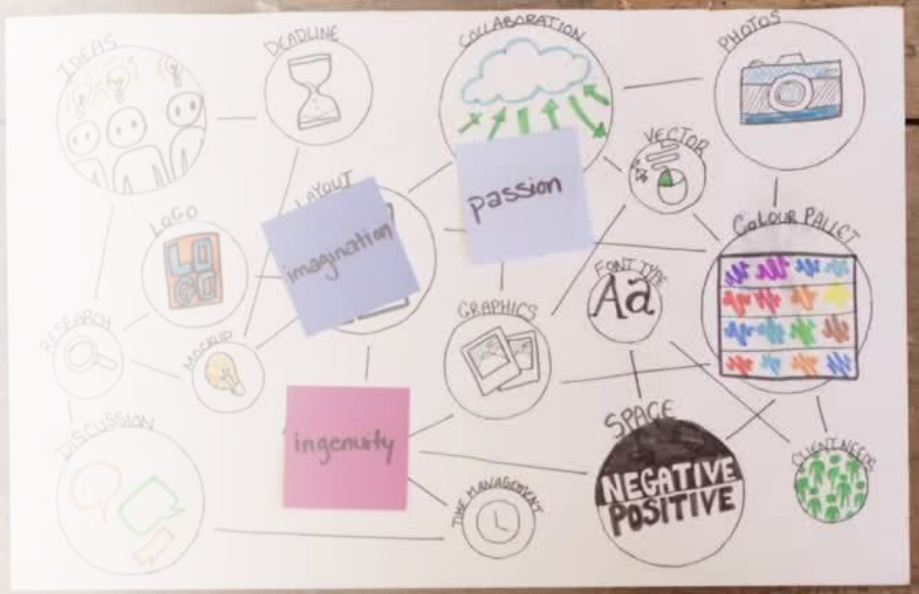
Student Course Registration System

Team members: Kyle Gosine, Weijun Huang, Mohamed Kamagate, Xinfan Li, and Hector Santana. CCNY - CSC 336



What’s Our Goal?

Our Student Course Registration System simplifies enrollment for students and helps administrators track their courses.

Students will be able to browse courses, register or drop classes, and view their schedules. Meanwhile, administrators will be able to efficiently manage their courses and student records.

We aim to improve organization, reduce errors, and provide a more convenient academic management experience for our users.

Technology Stack

Backend

Node.js + Express.js
For handling course search, enrollment, waitlisting, authentication, and role-based access control.

Frontend

React.js
Dynamic, real-time UI for students and instructors.
Fully responsive (mobile, tablet, desktop.)

Database

MySQL
Relational database for Students, Instructors, Courses, Enrollments, Waitlist.
Supports complex queries and real-time seat updates.

GitHub (Private / Public)
Version control and team collaboration.
Secure code management.

RESTful APIs · JWT Authentication · Postman Testing · Docker

Use Case 1: Course Search & Enrollment

User Journey

Student logs into the system → Student searches courses by department or keyword → System displays available sections

Student selects a section → System checks seat availability → Student enrolls successfully

System Actions

Retrieves course data from database → Updates enrollment table → Decrease available seats

Use Case 2: Instructor Schedule Management

User Journey

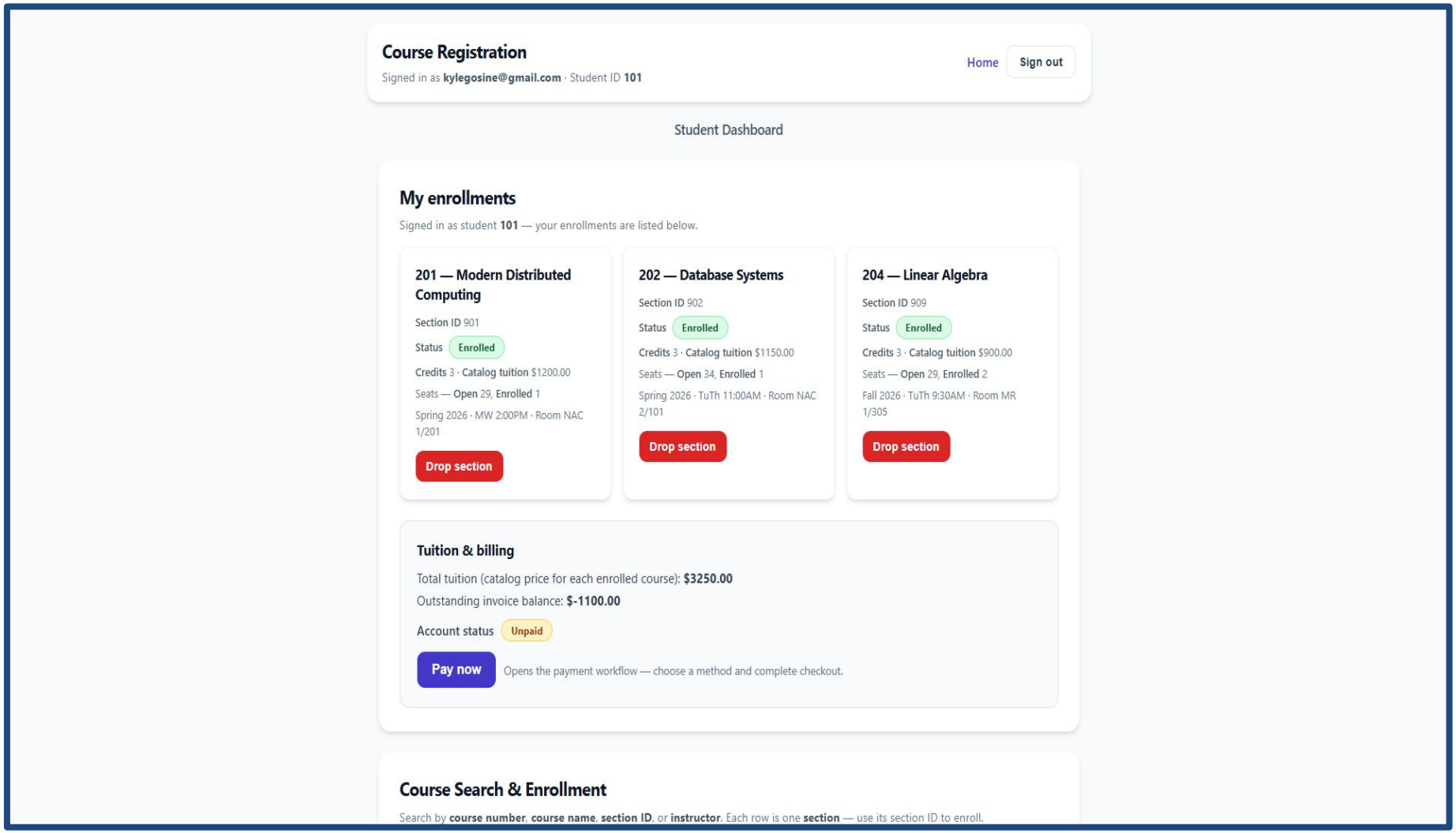
Instructor logs into the system → Instructor creates a course section

Instructor sets schedule and capacity → System publishes the section for students

System Actions

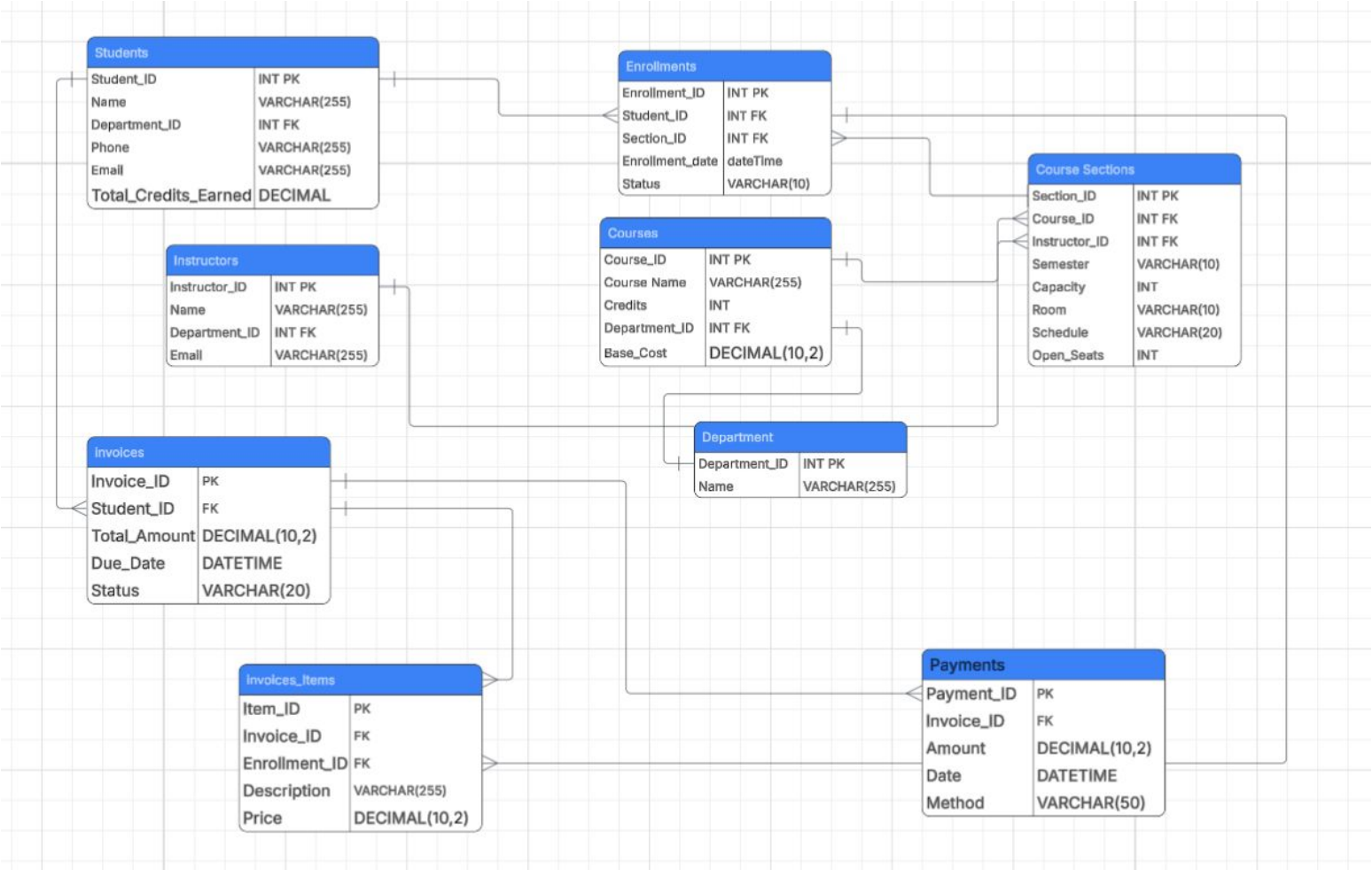
store section data in database → Validate schedule conflicts → Update course listing → Initialize enrollment counters

Interface DEMO



Feature DEMO

2NF Relational Model Diagram



Revenue Stream Reporting Interface

Range Year Chart months 12 Refresh Print / Save PDF Home

▶ Optional: secure revenue API for production

Total paid invoice revenue

$0

Sum of invoice totals marked Paid

— 0% vs prior period (payments)

Outstanding balance

$11,500

Invoices not yet fully collected

Active students

2

Distinct students with an active enrollment

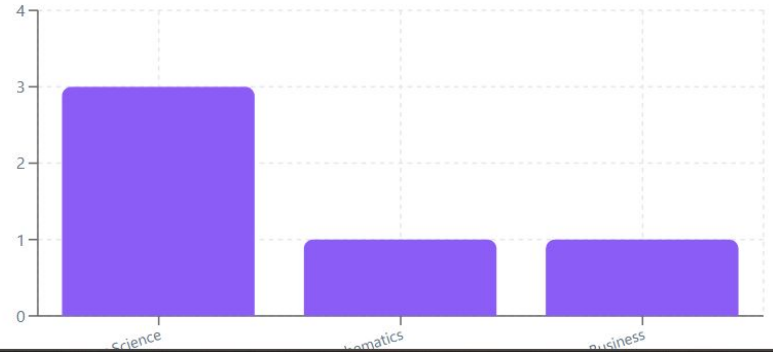
▲ +100% vs prior period (dated enrollments)

Payment inflow & cumulative

Tuition payments recorded by month (from Payments table)

Enrollments by department

Active enrollments linked to course department



Stored Procedures Demonstration

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `RegisterStudent`(IN p_student_id INT, IN p_section_id INT)
BEGIN
    DECLARE seats_left INT;

    -- Check if there are seats available
    SELECT Open_Seats INTO seats_left FROM Course_Sections WHERE Section_ID = p_section_id;

    IF seats_left > 0 THEN
        -- Add the enrollment
        INSERT INTO Enrollments (Enrollment_ID, Student_ID, Section_ID, Status)
        VALUES (NULL, p_student_id, p_section_id, 'Enrolled');

        -- Update the seats
        UPDATE Course_Sections SET Open_Seats = Open_Seats - 1 WHERE Section_ID = p_section_id;

        SELECT 'Registration Successful' AS Message;
    ELSE
        SELECT 'Registration Failed: Class Full' AS Message;
    END IF;
END ;;
```

Takeaway

Our biggest takeaway from this project was understanding how normalization actually make a project easier to build and improve.

At first, we wanted to track both enrolled and waitlisted students, but our table design made it too complex. After learning normalization, we better understood our limitations and how to structure our data, so we focused on enrollments for the sake of time.

Why this matters for future classes:

Proper normalization helps reduce confusion, avoid duplicate data, and turn complicated ideas into features that are easier to understand, manage, and implement.

Thanks